



**Gobierno Bolivariano
de Venezuela**

Ministerio del Poder Popular
para la Educación Universitaria

Universidad Nacional Experimental
para las Telecomunicaciones e Informática (UNETI)



REPÚBLICA BOLIVARIANA DE VENEZUELA

MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA

UNIVERSIDAD NACIONAL EXPERIMENTAL DE TELECOMUNICACIONES E INFORMÁTICA (UNETI)

PROGRAMA NACIONAL DE FORMACIÓN EN INFORMÁTICA

Evaluación 2

PARTICIPANTES:

C.I.: V-15.165.088 Jasmar Chirino

DOCENTE: Carlos Márquez

Caracas, junio de 2026

ÍNDICE GENERAL

1. Introducción.....	2
2. Reseña	3
3. Muestra del código.....	4
4. Conclusión	13
5. Referencia bibliográfica.....	14



INTRODUCCIÓN

Este trabajo presenta el diseño, desarrollo y resolución de problemas de una aplicación móvil de catálogo de productos utilizando la combinación de Angular e Ionic Framework. A lo largo del proyecto, se ha puesto en práctica la arquitectura de componentes standalone, la inyección de dependencias mediante servicios en memoria para la gestión de datos, y el sistema de enrutamiento dinámico para navegar entre listas y pantallas de detalle de forma fluida. Asimismo, se abordan aspectos clave de la maquetación móvil responsiva usando el sistema de rejillas (ion-grid) y la integración de formularios dinámicos. El objetivo principal de este proyecto es demostrar cómo la sinergia entre el robusto ecosistema lógico de Angular y los componentes visuales nativos de Ionic permite construir soluciones de e-commerce ágiles, escalables y visualmente atractivas.

[Aquí puede encontrar la aplicación actualizada:](#)

<https://github.com/Jasmar81/MyApp.git>

RESEÑA

Para construir la aplicación observamos los videos que se encuentran en la plataforma de la UNETI y por cada error o duda presentada me apoye en la IA GEMINI para corregir los errores o dudas que se iban generando:

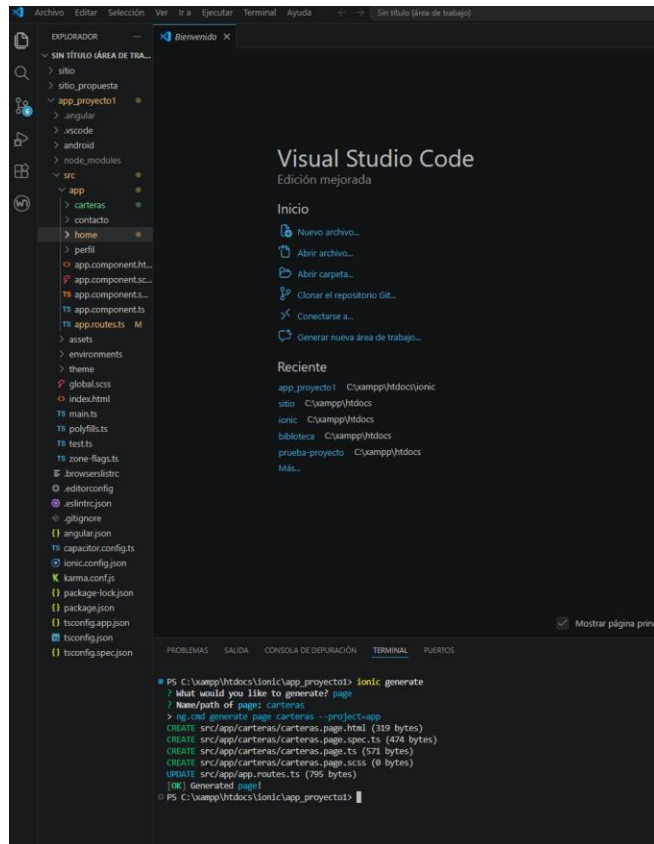
Como ya teníamos una app creada de la evaluación anterior se comenzaron a trabajar con esos archivos y actualizarlos según lo que iba viendo en el video. Ya casi para culminar luego de programar como eliminar un contenido, recurrí a la IA para crear el formulario de una vez, entendiendo que era una de las solicitudes de la evaluación.

Esta actualización me costó un poco, me costo entender que cada vez que se creaba un componentes personalizado de interfaz de usuario que proporciona el framework se debía colocar en los import en el .page.ts , de igual forma me complico un poco el tema de los nombres que le puse a los archivos cuando cree el servicio, ya sé que para otras oportunidades debo colocar nombres que me permitan diferenciar los archivos y no me generen un conflicto a la hora de crear el código.

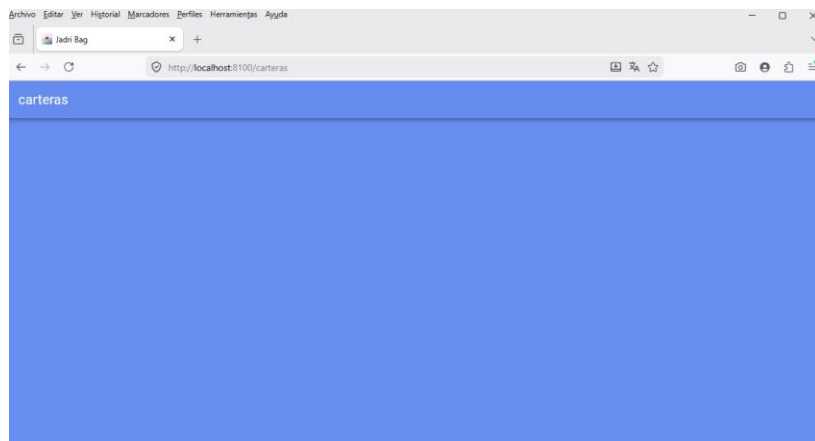
Para crear el formulario le pregunte a la IA con el siguiente prompt: ahora te pregunto cómo sería crear un formulario en angular e ionic; y ahí me arrojó todo el procedimiento a seguir.

MUESTRA DEL CÓDIGO

Se inicio el servidor en el editor:

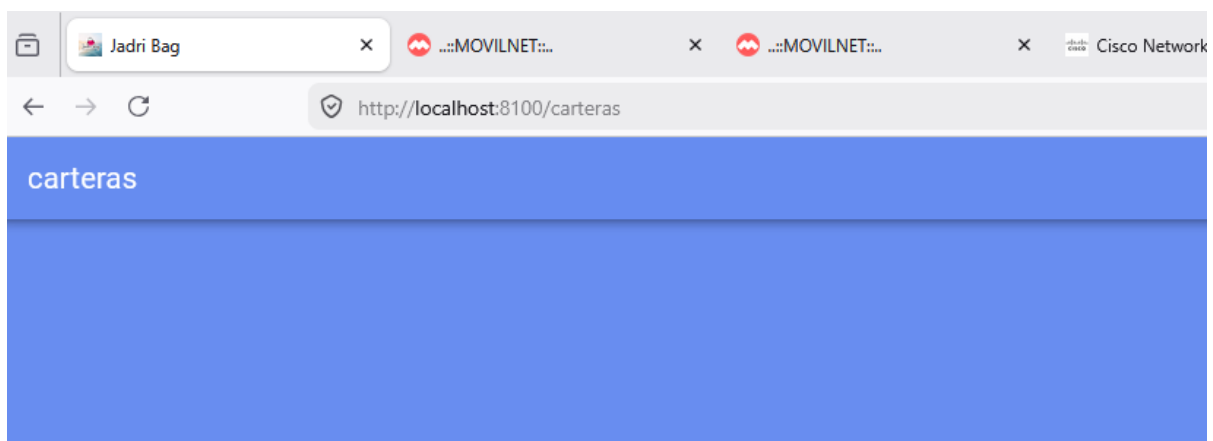


Frontend:



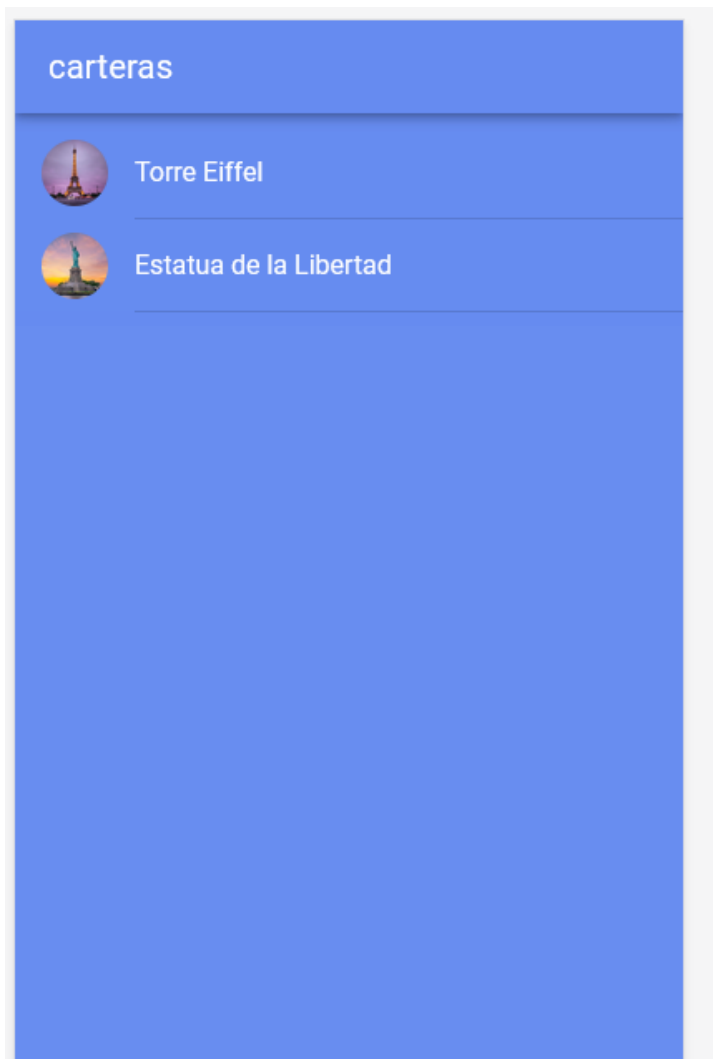
Como la versión es anterior me detuvo el pensar que estaba haciendo algo mal y resulta que el archivo del que habla el video es app.routes.ts

```
app_proyecto1 > src > app > TS app.routes.ts > routes > redirectTo
1 import { Routes } from '@angular/router';
2
3 export const routes: Routes = [
4   {
5     path: '',
6     redirectTo: 'carteras',
7     pathMatch: 'full',
8   },
9   {
10    path: 'home',
11    loadComponent: () => import('./home/home.page').then(m => m.HomePage),
12  },
13  {
14    path: 'perfil',
15    loadComponent: () => import('./perfil/perfil.page').then(m => m.PerfilPage)
16    // El ".then( m => m.PerfilPage)" debe ser igual al nombre de la clase en el archivo .
17  },
18  {
19    path: 'contacto',
20    loadComponent: () => import('./contacto/contacto.page').then(m => m.ContactoPage)
21  },
22  {
23    path: 'carteras',
24    loadComponent: () => import('./carteras/carteras.page').then(m => m.CarterasPage)
25  },
26 ]
```





Me costó que se viera como indicaba el video ya que no había actualizado los componentes Ion que estaba usando, avatar, list, title, al incluirlos en carteras.page.ts ya logre que se viera como se estaba mostrando en el video.





Se agrego el routerLink y se hizo mismo procedimiento se agregó a librerías en el .ts para que pudiese llamar y no dar error.

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { RouterLink } from '@angular/router';
import { IonContent, IonList, IonItem, IonAvatar, IonLabel, IonImg, IonHeader, IonToolbar, IonTitle } from '@ionic/angular/standalone';

@Component({
  selector: 'app-carteras',
  templateUrl: './carteras.page.html',
  styleUrls: ['./carteras.page.scss'],
  standalone: true,
  imports: [CommonModule, IonContent, IonHeader, IonToolbar, IonTitle, IonList, IonItem, IonAvatar, IonLabel, IonImg, RouterLink,]
})

export class CarterasPage implements OnInit {
  public Carteras = [
    {
      id: '1',

```

Tuve que investigar un poco porque pasa lo mismo anterior, por la actualización del video ya hay algunos componentes que no usa y tuve que colocar los actualizados, así quedo:

```
app.routes.ts
import { Routes } from '@angular/router';

export const routes: Routes = [
  {
    path: '',
    redirectTo: 'carteras',
    pathMatch: 'full',
  },
  {
    path: 'home',
    loadChildren: () => import('../home/home.page').then(m => m.HomePage),
  },
  {
    path: 'perfil',
    loadChildren: () => import('../perfil/perfil.page').then(m => m.PerfilPage),
  },
  {
    path: 'contacto',
    loadChildren: () => import('../contacto/contacto.page').then(m => m.ContactoPage),
  },
  {
    path: 'carteras',
    children: [
      {
        path: '',
        loadChildren: () => import('../carteras/carteras.page').then(m => m.CarterasPage),
      },
      {
        path: 'carterasid',
        loadChildren: () => import('../carteras/handoleras/handoleras.page').then(m => m.HandolerasPage),
      },
      {
        path: 'handoleras',
        loadChildren: () => import('../carteras/handoleras/handoleras.page').then(m => m.HandolerasPage),
      },
    ],
  },
];
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\xampp\htdocs\ionicapp_project> ionic serve

[ng] src/app/app.routes.ts:21:21

[ng] 21 |

[ng] > Changes detected. Rebuilding...

[ng] Initial chunk files

Files	Raw Size
styles.css	59.09 KB
main.js	2.96 KB
polyfills.js	143 bytes
Initial total	62.01 KB

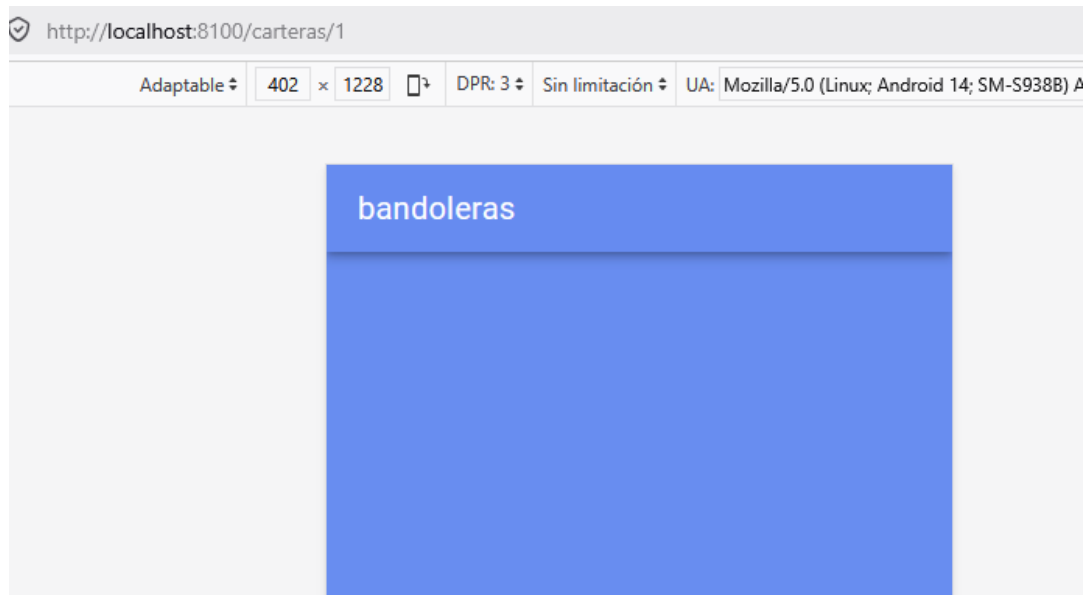
[ng] Lazy chunk files

Files	Raw Size
contacto.page-B2120W.js	6.74 KB
perfil.page-80V30W.js	6.28 KB
carteras.page-VQ9QWV.js	6.09 KB
home.page-5G8M5V.js	6.48 KB
handoleras.page-7AH8WQ.js	6.21 KB

[ng] Application bundle generation complete. [6.115 seconds] - 2025-06-04T15:42:31.666Z

[ng] Page reload sent to client(s).

Así se ve:



Luego de este proceso se creo un servicio que en ionic es una clase reutilizable encargada de realizar tareas específicas

```

PROBLEMAS 1 SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS
PS C:\xampp\htdocs\ionic\app_proyecto1> ionic serve

Local: http://localhost:8100

Use Ctrl+C to quit this process

[ng] → Local: http://localhost:8100/
[ng] → press h + enter to show help
[INFO] Browser window opened to http://localhost:8100!

[ng] ^C
● PS C:\xampp\htdocs\ionic\app_proyecto1> ionic generate
? What would you like to generate? service
? Name/path of service: carteras/carteras
> ng.cmd generate service carteras/carteras --project=app
CREATE src/app/carteras/carteras.spec.ts (347 bytes)
CREATE src/app/carteras/carteras.ts (121 bytes)
[OK] Generated service!
○ PS C:\xampp\htdocs\ionic\app_proyecto1>
    
```

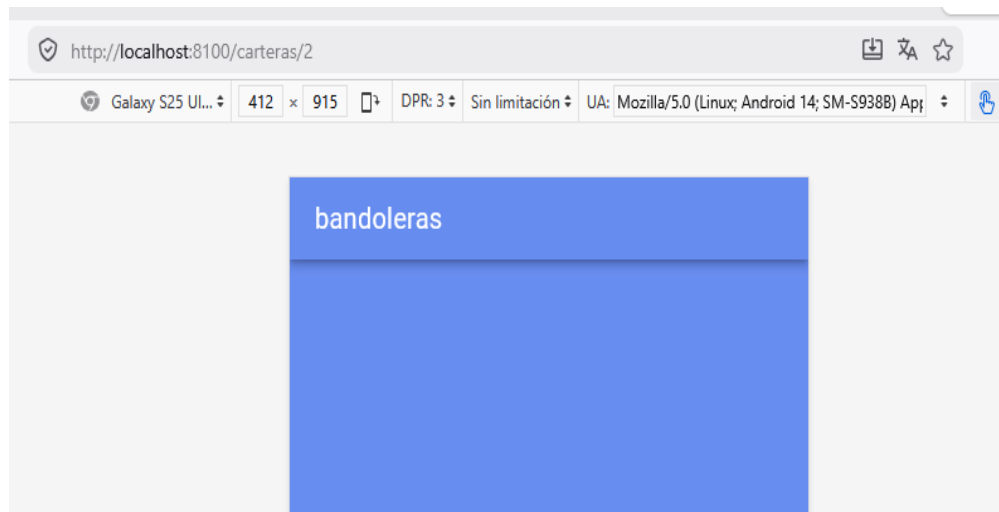
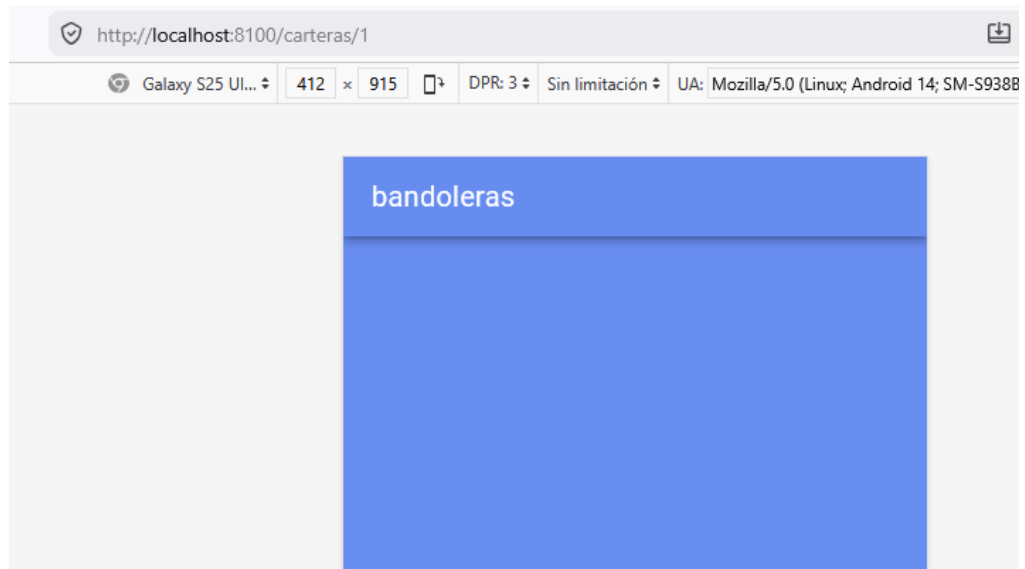


En la ruta carteras.service.ts así se ve el código:

```

6  export class CarterasService { // Ajustado el nombre de la clase
8      public carteras = [
16         id: '2',
17         title: 'Estatua de la Libertad',
18         imageURL: 'https://www.revistalimpiezas.es/wp-content/uploads/sites/4/2025/07/estatuadelalibertad.jpg',
19         comments: ['Un lugar maravilloso']
20     ]
21 };
22
23 // Corregido: Quitados los dos puntos ":" por llaves "{"
24 getCarteras() {
25     return [...this.carteras];
26 }
27
28 getCartera(carteraId: string) {
29     return {
30         ...this.carteras.find(cartera => {
31             return cartera.id === carteraId;
32         })
33     };
34 }
35
36 addCartera(title: string, imageURL: string) {
37     this.carteras.push({
38         title,
39         imageURL,
40         comments: [],
41         id: (this.carteras.length + 1).toString()
42     }); // Corregido el orden de cierre });
43 }
44
45 deleteCartera(carteraId: string) {
46     // Corregido: Asegurado que la variable coincida (carteraId)
47     this.carteras = this.carteras.filter(cartera => {
48         return cartera.id !== carteraId;
49     }); // Corregido el orden de cierre });
50 }
51 }
```

Ya el servicio funciona bien



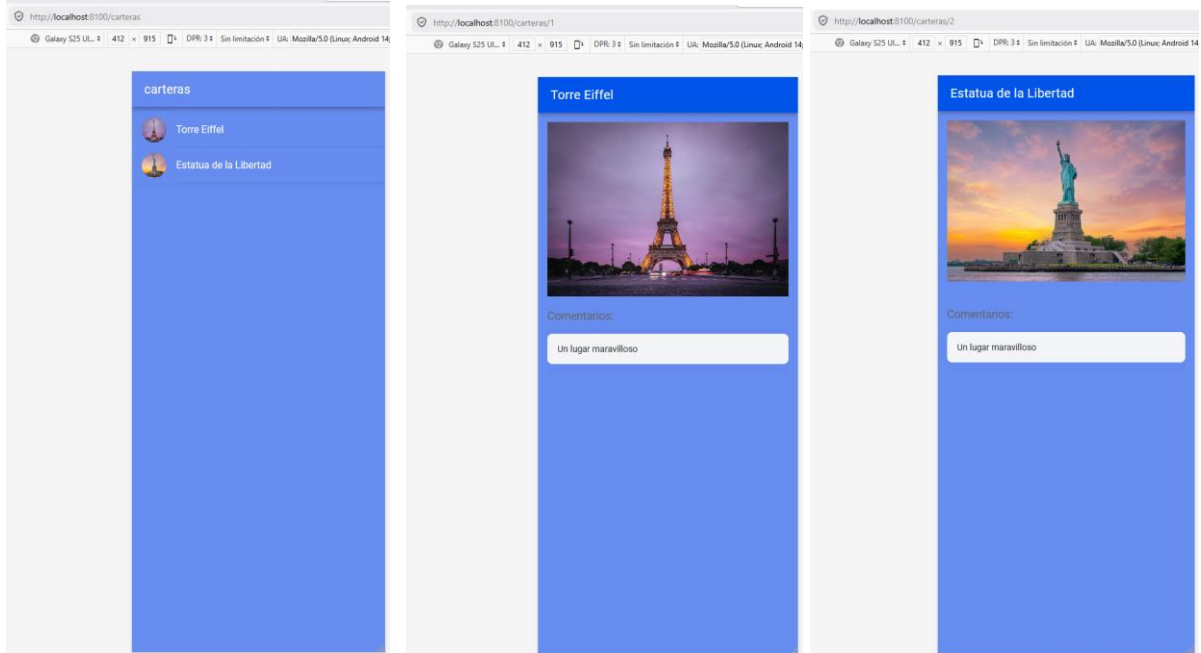


Ahora vamos a crear el enrutador para llamar a la otras páginas:

```
1 import { Component, OnInit } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { FormsModule } from '@angular/forms';
4 import { IonContent, IonHeader, IonTitle, IonToolbar } from '@ionic/angular/standalone';
5 import { ActivatedRoute } from '@angular/router';
6
7
8 @Component({
9   selector: 'app-bandoleras',
10  templateUrl: './bandoleras.page.html',
11  styleUrls: ['./bandoleras.page.scss'],
12  standalone: true,
13  imports: [IonContent, IonHeader, IonTitle, IonToolbar, CommonModule, FormsModule]
14 })
15 export class BandolerasPage implements OnInit {
16
17   constructor( private activatedRoute: ActivatedRoute ) { }
18
19   ngOnInit() {
20     this.activatedRoute.paramMap.subscribe(paramMap => {
21
22       if (!paramMap.has('carterasId')) {
23         return;
24       }
25
26       const recipeId = paramMap.get('carterasId');
27       console.log(recipeId);
28
29     });
30   }
31 }
32
33 }
```



Se logró que al darle clic en el nombre abra la otra página con la información que le agregamos:



Los archivos que debieron ser modificados fueron:

bandoleras.page.ts

```

TS bandoleras.page.ts U X  <> bandoleras.page.html U  TS app.routes.ts M  <> carteras.page.html U  TS carteras.service.ts U  ?  []  ...
app_proyecto1 > src > app > carteras > bandoleras > TS bandoleras.page.ts > BandolerasPage
1  import { Component, OnInit } from '@angular/core';
2  import { CommonModule } from '@angular/common';
3  import { FormsModule } from '@angular/forms';
4  import { IonContent, IonHeader, IonTitle, IonToolbar, IonImg, IonList, IonItem, IonLabel } from '@ionic/angular/standalone';
5  import { ActivatedRoute } from '@angular/router';
6  import { CarterasService } from '../carteras.service';
7
8
9  @Component({
10   selector: 'app-bandoleras',
11   templateUrl: './bandoleras.page.html',
12   styleUrls: ['./bandoleras.page.scss'],
13   standalone: true,
14   imports: [ CommonModule,
15     FormsModule,
16     IonContent,
17     IonHeader,
18     IonTitle,
19     IonToolbar,
20     IonImg,
21     IonList,
22     IonItem,
23     IonLabel ]
24 })
25 export class BandolerasPage implements OnInit {
26
27   // Variable donde guardaremos los datos de la cartera actual
28   public carteras: any;
29
30   constructor(
31     private activatedRoute: ActivatedRoute,
32     private carterasService: CarterasService
33   ) {}
34
35   ngOnInit() {
36     this.activatedRoute.paramMap.subscribe(paramMap => {
37
38       // Intentamos leerlo de las dos formas posibles:
39       const id = paramMap.get('carteraId') || paramMap.get('carterasid');
40
41       console.log('ID recuperado con éxito:', id);
42
43       if (id) {
44         this.carteras = this.carterasService.getCartera(id);
45         console.log('Datos de la cartera cargados:', this.carteras);
46       } else {
47         console.error('No se pudo encontrar ningún ID en la URL. Revisa tus rutas.');

```



app.routes.ts

```
TS bandoleras.page.ts U  bandoleras.page.html U  TS app.routes.ts M X  carteras.page.html U  TS carteras.service.ts U  ...
app_proyecto1 > src > app > TS app.routes.ts > routes > children > loadComponent
1  import { Routes } from '@angular/router';
2
3  export const routes: Routes = [
4  {
5    path: '',
6    redirectTo: 'carteras',
7    pathMatch: 'full',
8  },
9  {
10   path: 'home',
11   loadComponent: () => import('./home/home.page').then((m) => m.HomePage),
12 },
13 {
14   path: 'perfil',
15   loadComponent: () => import('./perfil/perfil.page').then((m) => m.PerfilPage),
16 },
17 {
18   path: 'contacto',
19   loadComponent: () => import('./contacto/contacto.page').then((m) => m.ContactoPage),
20 },
21 {
22   path: 'carteras',
23   children: [
24     {
25       path: '',
26       loadComponent: () => import('./carteras/carteras.page').then((m) => m.CarterasPage),
27     },
28     {
29       path: ':carterasId',
30       loadComponent: () => import('./carteras/bandoleras/bandoleras.page').then((m) => m.BandolerasPage),
31     },
32   ],
33   path: 'carteras/:bandoleraId', // <-- El ":carterasId" es vital. Debe coincidir con lo que buscas en tu
34   paramMap.get()
35   loadComponent: () => import('./carteras/bandoleras/bandoleras.page').then( m => m.BandolerasPage)
36 },
37 ],
38 ];
```




En este vemos como se muestra en el backend (bandoleras.page.html):

```
TS bandoleras.page.ts U  bandoleras.page.html U X  TS app.routes.ts M  carteras.page.html U  TS carteras.service.ts  ...
app_proyecto1 > src > app > carteras > bandoleras > bandoleras.page.html > ion-content
1  <ion-header [translucent]="true">
2    <ion-toolbar color="primary">
3      <ion-title>{{ carteras?.title }}</ion-title>
4    </ion-toolbar>
5  </ion-header>
6
7  <ion-content [fullscreen]="true">
8    <ion-header collapse="condense">
9      <ion-toolbar>
10        <ion-title size="large">{{ carteras?.title }}</ion-title>
11      </ion-toolbar>
12    </ion-header>
13
14    <div style="text-align: center; padding: 15px;">
15      <ion-img [src]="carteras?.imageUrl" style="max-height: 250px; border-radius: 8px; object-fit: cover;">
16    </div>
17
18    <div style="padding: 15px;">
19      <h3 style="color: #666; font-size: 1.1em; margin-bottom: 10px;">Comentarios:</h3>
20      <ion-list lines="none">
21        <ion-item *ngFor="let comentario of carteras?.comments" style="--background: #f4f5f8; margin-bottom: 8px;
22          border-radius: 8px;">
23          <ion-label>
24            <p style="color: #333; margin: 0;">{{ comentario }}</p>
25          </ion-label>
26        </ion-item>
27      </ion-list>
28    </div>
29  </ion-content>
```



Ahora vamos a agregar un botón de atrás:

Se agrego el código en el archivo bandoleras.page.html:

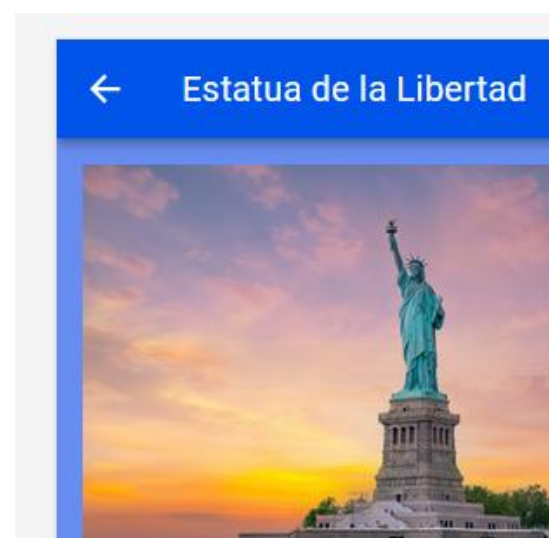
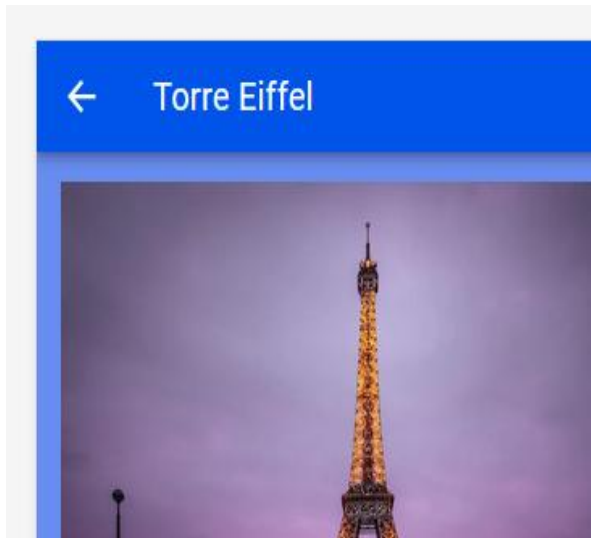
```
app_proyecto1 > src > app > carteras > bandoleras > bandoleras.page.html > ion-header > ion-toolbar >
1 <ion-header [translucent]="true">
2
3 <ion-title>{{ carteras?.title }}</ion-title>
4 <ion-buttons slot="start">
5   <ion-back-button defaultHref="/carteras"></ion-back-button>
6 </ion-buttons>
7 </ion-header>
8 <ion-header>
9
10 <ion-content [fullscreen]="true">
11 <ion-header collapse="condense">
12 <ion-toolbar>
13   <ion-title size="large">{{ carteras?.title }}</ion-title>
14 </ion-toolbar>
15 </ion-header>
16
17 <div style="text-align: center; padding: 15px;">
18   <ion-img [src]="carteras?.imageUrl" style="max-height: 250px; border-radius: 8px;">
19 </div>
20
21 <div style="padding: 15px;">
22   <h3 style="color: #666; font-size: 1.1em; margin-bottom: 10px;">Comentarios:
23   <ion-list lines="none">
24     <ion-item *ngFor="let comentario of carteras?.comments" style="--background:
25       <ion-label>
26         <p style="color: #333; margin: 0;">{{ comentario }}</p>
27       </ion-label>
28     </ion-item>
29   </ion-list>
30 </div>
```

Y en bandoleras.page.ts

```
app_proyecto1 > src > app > carteras > bandoleras > bandoleras.page.ts >
1 import { Component, OnInit } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3
4 import { IonContent, IonHeader, IonTitle, IonToolbar, IonImg, IonList, IonItem, IonLabel, IonButtons, IonBackButton } from '@ionic/angular/standalone';
5 import { ActivatedRoute } from '@angular/router';
6 import { CarterasService } from '../carteras.service';
7
8
9
10 @Component({
11   selector: 'app-bandoleras',
12   templateUrl: './bandoleras.page.html',
13   styleUrls: ['./bandoleras.page.scss'],
14   standalone: true,
15   imports: [ CommonModule,
16     FormsModule,
17     IonContent,
18     IonHeader,
19     IonTitle,
20     IonToolbar,
21     IonImg,
22     IonList,
23     IonItem,
24     IonLabel,
25     IonButtons,
26     IonBackButton ]
27 })
28 export class BandolerasPage implements OnInit {
29
30 }
```



Así quedaron los botones de ir a atrás:





Comenzando a personalizar la aplicación:

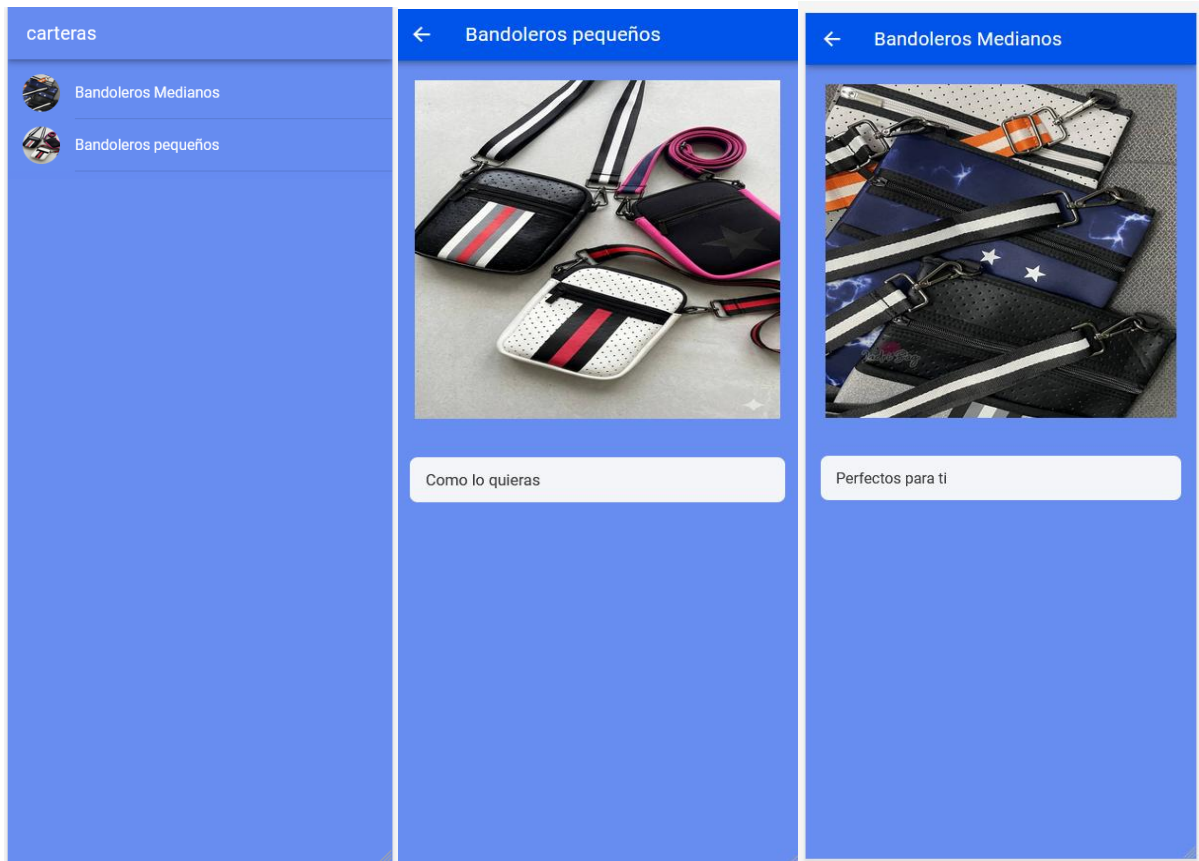
El código:

```

1  <ion-header [translucent]="true">
2  <ion-toolbar color="primary">
3  <ion-buttons slot="start">
4  <ion-back-button defaultHref="/carteras"></ion-back-button>
5  </ion-buttons>
6  <ion-title>{{ Carteras?.title }}</ion-title>
7  </ion-toolbar>
8  </ion-header>
9
10 <ion-content [fullscreen]="true">
11 <ion-header collapse="condense">
12 <ion-toolbar>
13 <ion-title size="large">{{ Carteras?.title }}</ion-title>
14 </ion-toolbar>
15 </ion-header>
16
17 <ion-grid fixed style="padding: 0;">
18
19   <ion-row>
20     <ion-col size="12">
21       <div style="text-align: center; padding: 16px; ">
22         <ion-img
23           [src]="Carteras?.imageUrl"
24           style="max-height: 40vh; width: 100%; object-fit: contain; border-radius: 12px; display: block;
25           margin: 0 auto;">
26         </ion-img>
27       </div>
28     </ion-col>
29   </ion-row>
30
31   <ion-row>
32     <ion-col size="12" style="padding: 20px 16px;">
33       <!-- <h3 style="color: #666; font-size: 1.2em; margin: 0 0 12px 0; font-weight: 600;">
34         Comentarios:
35       </h3> -->
36
37       <ion-list lines="none" style="padding: 0; background: transparent;">
38         <ion-item *ngFor="let comentario of Carteras?.comments" style="--background: #f4f5f8; margin-bottom:
39         8px; border-radius: 8px;">
40           <ion-label>
41             <p style="color: #333; margin: 0; font-size: 1em;">{{ comentario }}</p>
42           </ion-label>
43         </ion-item>
44       </ion-list>
45     </ion-col>
46   </ion-row>
47 </ion-grid>
</ion-content>

```

La aplicación:



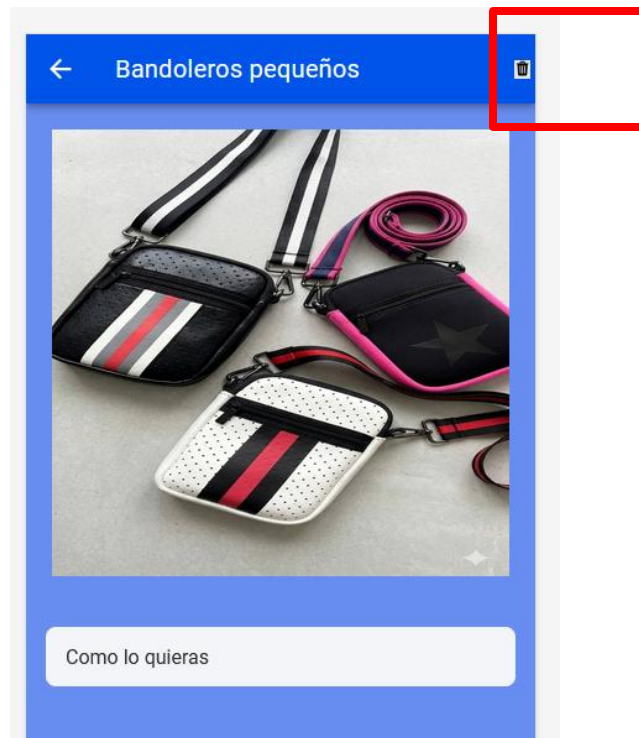


Ahora muestro como aprendí a eliminar datos en la aplicación:

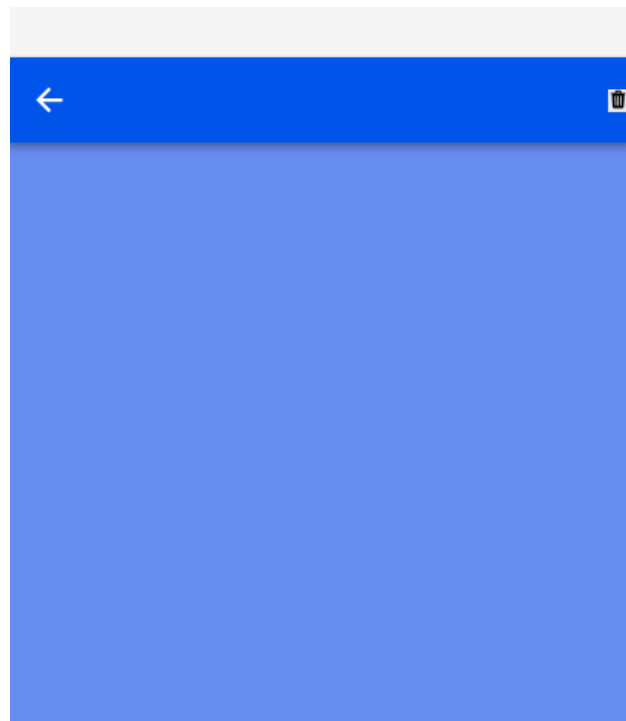
Seguí el video para buscar el componente de eliminar en la página de angular

Ya se agrego en el archivo bandoleras.page.html para eliminar un contenido:

```
carteras.page.html U  TS bandoleras.page.ts U  bandoleras.page.html U X  TS Carteras.service.ts U
app_proyecto1 > src > app > Carteras > bandoleras > bandoleras.page.html > ...
1
2 <ion-header [translucent]="true">
3   <ion-toolbar color="primary">
4
5     <ion-buttons slot="start">
6       <ion-back-button defaultHref="/Carteras"></ion-back-button>
7     </ion-buttons>
8
9     <ion-title>{{ Carteras?.title }}</ion-title>
10
11     <ion-buttons slot="end">
12       <button ion-button icon-only (click)="eliminarCartera()">
13         <ion-icon name="trash"></ion-icon>
14       </button>
15     </ion-buttons>
16
17   </ion-toolbar> </ion-header>
18
19 <ion-content [fullscreen]="true">
20   <ion-header collapse="condense">
21     <ion-toolbar>
22       <ion-title size="large">{{ Carteras?.title }}</ion-title>
23     </ion-toolbar>
24   </ion-header>
25
26   <ion-grid fixed style="padding: 0;">
27
28     <ion-row>
29       <ion-col size="12">
30         <div style="text-align: center; padding: 16px; ">
31           <ion-img
32             [src]="Carteras?.imageUrl"
33             style="max-height: 40vh; width: 100%; object-fit: contain; border-radius: 12px; display: block; margin: 0 auto;
34           </ion-img>
35         </div>
36       </ion-col>
37     </ion-row>
```



Al darle clic en el bote de eliminar borra el contenido:



Como no tenemos una base de datos al darle actualizar al navegador el contenido regresa sin perderse.

Ahora vamos a crear un formulario con angular e ionic:

Como hacer todo lo anterior me consumió mucho tiempo, el formulario lo cree preguntando a la IA como podría crearlo, lo hice con el siguiente prompt: **ahora te pregunto cómo sería crear un formulario en angular e ionic.** Aquí me indicó como hacerlo creando las etiquetas especiales: Ionic diseñadas para que se sientan como una aplicación móvil real. Las más comunes son: `<ion-input>`: Para cajas de texto normales (como el título de la cartera o el precio). `<ion-textarea>`: Para textos más largos (como la descripción o los comentarios). `<ion-button>`: El botón para enviar los datos. Luego, Con los formularios reactivos, tú creas una estructura en tu archivo TypeScript (.ts) que vigila exactamente qué pasa en cada casilla.

`FormControl`: Controla una sola casilla de texto (sabe qué tiene escrito y si es válido).

`FormGroup`: Agrupa todas las casillas (por ejemplo: el conjunto de Título, Imagen y Comentario).

Esto me permitió crear el formulario y se ve así en el frontend:

← Bandoleros Medianos

Nombre de la Cartera

Enlace de la Imagen (URL)

AGREGAR CARTERA

Perfectos para ti



El código bandoleras.page.ts:

```

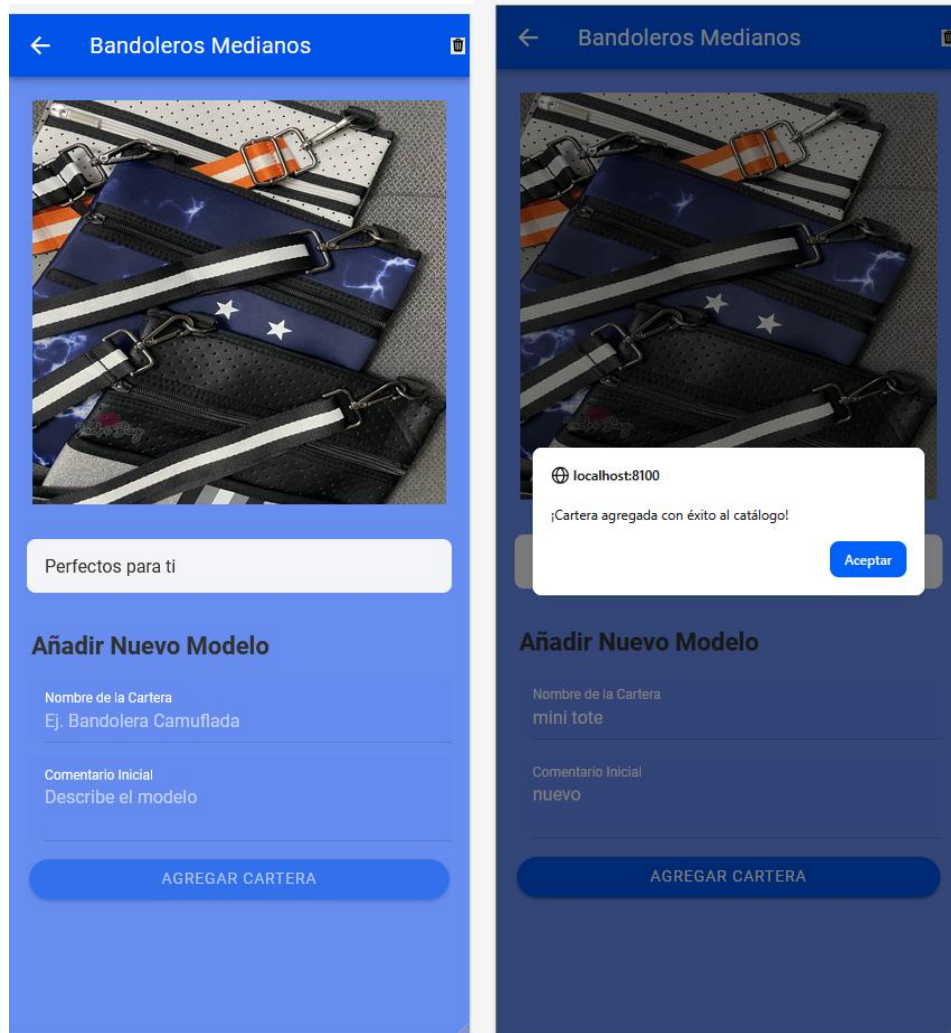
carteras.page.html U  TS bandoleras.page.ts U X  bandoleras.page.html U  TS Carteras.service.ts U
app.proyecto1 > src > app > Carteras > bandoleras > TS bandoleras.page.ts > CarterasPage > guardarCartera
26 export class BandolerasPage implements OnInit {
39   ngOnInit() {
47     this.activatedRoute.paramMap.subscribe(paramMap => {
49       console.log('ID recuperado con éxito:', id);
50     });
51     if (id) {
52       this.carteras = this.carterasService.getCartera(id);
53       console.log('Datos de la cartera cargados:', this.carteras);
54     } else {
55       console.error('No se pudo encontrar ningún ID en la URL.');
```

carteras.service.ts

```

carteras.page.html U  TS bandoleras.page.ts U  bandoleras.page.html U  TS Carteras.service.ts U X
app.proyecto1 > src > app > Carteras > TS Carteras.service.ts > CarterasService > addCartera
6 export class CarterasService { // Ajustado el nombre de la clase
22
23
24   getCarteras() {
25     return [...this.carteras];
26   }
27
28   getCartera(carteraId: string) {
29     return this.carteras.find(cartera => {
30       return carterId === carterId;
31     });
32   }
33
34   // Agrega esta función dentro de la clase de tu CarterasService
35   addCartera(title: string, imageURL: string, comment: string) {
36     const nuevaCartera = {
37       id: (this.carteras.length + 1).toString(), // Le genera un ID automático (Ej: "3")
38       title: title,
39       imageURL: imageURL,
40       comments: comment ? [comment] : [] // Guarda el comentario en un arreglo de textos
41     };
42
43     this.carteras.push(nuevaCartera); // Inserta el nuevo objeto dentro del arreglo real
44     console.log('Arreglo actualizado en el servicio:', this.carteras);
45   }
46
47   deleteCartera(carteraId: string) {
48     // Corregido: Asegurado que la variable coincida (carteraId)
49     this.carteras = this.carteras.filter(cartera => {
50       return carterId !== carterId;
51     });
52   }
53
54
55
56 }
```

Ahora procedo a pulir el formulario para que quede toda la información ordenada, además voy a programar para que al crear la cartera quede registrada en la aplicación:





CONCLUSIÓN

La realización de este proyecto permite concluir que la integración de Angular e Ionic reduce drásticamente la brecha entre el desarrollo web y el desarrollo móvil nativo. A través de la implementación práctica, se logró consolidar el flujo completo de una aplicación: desde la visualización dinámica de datos mediante directivas estructuradas (*ngFor), pasando por la gestión lógica a través de controladores en TypeScript, hasta la depuración avanzada de errores comunes de compilación relacionados con la jerarquía visual de las etiquetas de Ionic.

Una de las lecciones más valiosas del proceso fue comprender la importancia de la arquitectura modular y el control estricto del ciclo de vida de los componentes (ngOnInit), lo cual garantiza que la información se transmita de forma correcta entre pantallas. Aunque la aplicación actualmente opera con un servicio de almacenamiento temporal en memoria, la estructura desarrollada queda perfectamente preparada para escalar hacia una base de datos persistente en la nube. En definitiva, las competencias adquiridas en este trabajo sientan las bases fundamentales para afrontar con éxito el desarrollo de aplicaciones móviles comerciales modernas, eficientes y centradas en el usuario.



REFERENCIA BIBLIOGRÁFICA

- Google. (2026). *Gemini* (Versión 3.0 Flash) [coloco recipeld y me trae nullo que puede ser]. <https://gemini.google.com/>
- Google. (2026). *Gemini* (Versión 3.0 Flash) [ya me funciona bien, pero debería de llamarme al torre ieffel y el botón de abajo a estatua de la libertad pero los 2 me llaman a torre eiffel que puede hacer en el código]. <https://gemini.google.com/>
- Universidad Nacional Experimental de las Telecomunicaciones e Informática. (2026). Título de la Actividad o Recurso [PROG_III_MOD_2-6A-B-7A-B-2026-1, Sesión Didáctica 2, Aplicación Práctica]. Campus Virtual UNETI. <https://www.uneti.edu.ve/campus/>
- Ionic4 & Angular | Curso Práctico para principiantes, Parte 1 (24 de diciembre de 2019). [Video]. YouTube.
<https://youtu.be/MMRqyQVQ980?si=EsJH1GUJ0lsIu3YQ>
- Ionic4 & Angular | Curso Práctico para principiantes, Parte 2. (24 de diciembre de 2019). [Video]. YouTube.
https://youtu.be/bJ_DALcR1Ts?si=yKYO0uIcqtmWJQmM